

CHOICE Forge — Layer 3 Explainer

What T5 actually does: the encoder-decoder model itself, fine-tuning a pretrained checkpoint, role-prefix conditioning, why one pooled model beats nine, and where synthesis confidence comes from.

Source line from the architecture doc: [Layer 3] T5-small, fine-tuned on this dataset (train_seq2seq.py -> value_synthesizer/) → takes the span(s) Layer 2 found and generates the canonical value. One pooled model handles all 9 roles via a role-prefix instruction, standard T5-style multi-task conditioning.

LAYER 3 • CONCEPT 1

T5 (Text-to-Text Transfer Transformer) (the model itself)

In plain terms

The CRF only ever points at words already in the sentence — a highlighter. T5 actually writes new text as its answer, like a mini translator: instead of translating between languages, it's "translating" a raw extracted phrase into its clean, canonical form.

Technically

T5 treats every task as text-in, text-out. It's an encoder-decoder transformer — the encoder reads the input and builds a representation of it, the decoder generates output tokens one at a time, each new token conditioned on both the encoder's representation and everything decoded so far (autoregressive generation). `t5-small` is a pretrained checkpoint (~60M params) already trained on a huge generic corpus before you touch it — fine-tuning adapts it to your narrow task instead of learning language from scratch.

```
from transformers import T5Tokenizer, T5ForConditionalGeneration
```

```
tokenizer = T5Tokenizer.from_pretrained("t5-small")
model = T5ForConditionalGeneration.from_pretrained("t5-small")
```

Why it matters in this pipeline

This is the fundamental reason Layer 3 exists at all. It's the only component in the pipeline capable of producing "reduce churn" from "Get monthly logo churn under 2%" — text that never literally appeared in the input.

LAYER 3 • CONCEPT 2

Fine-tuning a pretrained model

In plain terms

Instead of teaching a student English from zero, you take a student who already speaks English fluently and just teach them your company's specific vocabulary and formatting habits — far faster, and needs far less

data than starting from nothing.

Technically

Fine-tuning continues training from the pretrained weights on your small task-specific dataset (629 pairs), at a much lower learning rate, for enough epochs to adapt without destroying its general language ability. One detail worth understanding in `preprocess()`: padding tokens in the target labels get replaced with `-100`, a reserved value that tells the loss function “ignore this position” — otherwise the model would waste capacity learning to predict padding as if it were real content.

```
def preprocess(batch):
    model_inputs = tokenizer(batch["input"], max_length=64,
                             truncation=True, padding="max_length")
    labels = tokenizer(text_target=batch["target"], max_length=32,
                       truncation=True, padding="max_length")
    labels["input_ids"] = [
        [(t if t != tokenizer.pad_token_id else -100) for t in seq]
        for seq in labels["input_ids"]
    ]
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs
```

Why it matters in this pipeline

This connects directly to the honest numbers reported earlier — train loss ≈ 0.83 , eval loss ≈ 3.1 . That gap is the fine-tuning process visibly straining against too little data: the model is memorizing the 535 training pairs faster than it's learning patterns that generalize to the 94 it hasn't seen. More labeled rows narrows that gap over time; it's a data-volume signal to track, not a bug to patch.

Role-prefix instruction / multi-task conditioning

In plain terms

Telling the model, at the start of every input, what kind of task this is — like handing an assistant a short instruction before each request: “translate this,” “summarize this,” “normalize this actor field” — so one assistant can act like nine specialists.

Technically

T5's original pretraining already used this exact pattern ("translate English to German: ...", "summarize: ...") to condition one shared model on many tasks. `build_seq2seq_pairs.py` reuses it directly. The [NEGATED] fix from the earlier negation bug is the same mechanism applied a second time, stacked on top — an explicit instruction rather than expecting the model to infer negation implicitly from a rare surface pattern.

```
pairs.append({
    "id": row["id"], "role": r,
    "input": f"normalize {r}: {src}",
    "target": val,
})

# with the negation fix layered on top:
# "normalize [NEGATED] constraints: without increasing the ad spend budget"
```

Why it matters in this pipeline

The prefix is the only thing that tells one set of shared weights “clean this up as an actor” versus “clean this up as a magnitude” — without it, the model has no way to know which of nine very different normalization styles to apply to a given span.

Pooling instead of nine separate models

In plain terms

Rather than training nine separate students, each with only a handful of practice problems (some roles have just 25–38 examples), you train one student on all the practice problems combined, with a note at the top of each problem saying which subject it belongs to. That student gets far more total practice and can carry skills learned from one subject over to another.

Technically

Per-role example counts range from 25 (actor) to 100 (intent) — nine independent T5 instances would leave the thinnest roles badly under-trained. Pooling into one model gives it 629 total examples instead of a handful per model, and lets shared parameters transfer general “strip filler words, canonicalize phrasing” ability across roles — something learned from cleaning up a scope span (dropping a leading “the”) can help the model clean up an object span too, even if that exact pattern was rarer in object's own training data.

```
# one shared model, conditioned by prefix, instead of:
#   actor_model.fit(25 examples)      <- badly under-trained
#   constraints_model.fit(38 examples) <- badly under-trained
#   ... x9
#
# becomes:
#   shared_model.fit(629 examples, role given via prefix)
```

Why it matters in this pipeline

This is a direct data-scarcity mitigation, not just a convenience. Without pooling, actor (25 examples) and constraints (38 examples) would likely underperform far worse than they already do.

Generation at inference, and synthesis confidence

In plain terms

Once trained, producing an answer means the model writes one word at a time, each time looking at what it's already written plus the instruction, deciding whether to keep going or stop.

Technically

`model.generate()` performs autoregressive decoding — greedy here, though beam search or sampling are alternatives — producing tokens until an end-of-sequence token or `max_new_tokens` is reached. `pipeline.py`'s `synthesize()` also requests `output_scores=True`, giving per-step token probability distributions that get averaged into a synthesis confidence number, separate from (and multiplied against) the CRF's extraction confidence.

```
def synthesize(self, role, span_text):
    prefix = "normalize [NEGATED] " if has_negation(span_text) else "normalize "
    prompt = f"{prefix}{role}: {span_text}"
    inputs = self.tokenizer(prompt, return_tensors="pt")
    out = self.t5.generate(
        **inputs, max_new_tokens=32,
        output_scores=True, return_dict_in_generate=True,
    )
    value = self.tokenizer.decode(out.sequences[0], skip_special_tokens=True)
```

Why it matters in this pipeline

This is the second half of the confidence number in the output table — extraction confidence (CRF marginals) × synthesis confidence (T5 generation probability), before the polarity guard applies its own caps on top.

Quick reference

Concept	Where it lives	Input	Output
T5 model	<code>value_synthesizer/</code> (<code>train_seq2seq.py</code>)	role-prefixed span text	generated canonical value
Fine-tuning	<code>Seq2SeqTrainer.train()</code>	629 (input, target) pairs	adapted model weights
Role-prefix conditioning	"normalize {role}: {span}"	role name + span	task-specific normalization
Pooling	single shared model, all roles	629 pairs across 9 roles	shared "cleanup" ability
Generation + confidence	<code>synthesize()</code> (<code>pipeline.py</code>)	prompt string	value + avg token probability